

Dungeon Crawler RPG — 프로젝트 요약

GIST AI2000 자료구조및알고리즘 / 팀 프로젝트 / Evaluation Guidance(200점) 대응 요약

1. 게임 개요

Python + pygame로 구현한 절차적 던전 크롤러 RPG. 플레이어는 절차 생성된 던전을 탐험하며 적과 턴 기반 전투를 하고, 아이템을 모으고, 층을 내려가 5층 보스를 처치한다. 게임 종료 시 점수가 리더보드에 기록된다. 게임 루프 / 렌더링 / 입력은 main.py가, 상태는 매 턴 하나의 Frame(player / map / enemies / items 묶음) 스택으로 관리한다.

2. 6대 핵심 기능 < - > 자료구조/알고리즘 (Guidance 요구 형식)

Guidance는 기능마다 ① 사용한 DS/알고리즘 ② 선택 이유 ③ 대안 비교 ④ 복잡도 ⑤ 코드 위치를 답하도록 요구한다 (6 features x 15점 + 인터뷰 25%).

기능	DS / 알고리즘	선택 이유 / 대안 비교	복잡도	코드 위치
1. Dungeon Map	2D 그리드 + 절차 생성 (방 배치 + L자 복도)	그리드는 좌표 -> 타일 O(1) 접근. 방 겹침은 AABB로 검사. 그래프(인접리스트) 대비 격자 이동 / 렌더에 단순 / 직관적.	접근 O(1) 생성 O(W / H)	map.py Map, Room _generate / _connect_rooms
2. Undo System	Stack (LIFO) — 프레임 히스토리	되돌리기는 LIFO이므로 Stack이 정확히 맞음. Queue(FIFO)면 순서가 거꾸로 됨. 전체 상태 deepcopy 저장(델타 아님) — 단순하지만 메모리 증가.	push/pop O(1) 최대 30프레임	frame.py Frame (list 기반) main.py:63 deepcopy
3. Turn Management	순차 턴 루프 (입력 -> 플레이어 -> 적 -> 픽업)	현재 모든 캐릭터 동일 속도라 단순 순회로 충분. 속도가 다르면 Priority Queue(Heap)로 행동 순서 정렬이 더 적합.	턴당 O(적 수)	main.py:52 turn_update()
4. Item Inventory	List (고정 10칸) + 아이템 클래스 계층	슬롯 수가 작고 1~0키로 인덱스 접근하므로 List가 단순 / 충분. 이름 검색이 잦으면 Dict, 정렬 / 범위검색이 필요하면 Tree가 유리.	추가/접근 O(1) 검색 O(n)	player.py inventory use_item() item.py Item/Weapon/Potion
5. Enemy AI	BFS 추적 + Bresenham 시야 + 도주	격자가 무가중치라 BFS가 최단경로를 보장하며 구현이 단순. A*는 휴리스틱으로 더 빠르나 가중치/큰 맵에서 이점 — 현 규모엔 과함.	BFS O(V+E) 시야 O(거리)	enemy.py _bfs_next_step() can_see (Bresenham)
6. Leaderboard	Dict 저장 + sorted() 내림차순 정렬	이름 -> 최고점 매핑은 Dict가 적합. 전체 정렬은 O(n log n). top-k만 필요하면 Heap이 O(n log k)로 유리.	정렬 O(n log n) 조회 O(1)	leaderboard.py add_score / get_leaderboard render.py _compute_score

3. 인게임 점수 공식

점수 = 킬XP x 10 + 골드(아이템) x 500 + Undo 잔여 x 90 + max(0, 3000 - 경과초) x 10 (render.py 의 _compute_score 함수)

4. 제출 / 평가 대비 체크리스트 (Guidance 기준)

항목 (배점)	준비 내용
PPT & 알고리즘 설명 제출 (30%)	전체 구조 + 6기능 + DS/알고리즘 + 선택이유 + 대안비교 + 코드위치 포함
6대 핵심 기능 구현 (15%)	위 2번 표의 6기능 모두 코드에 실재 — 대부분 구현 완료
알고리즘 비교 / 설명 (10%)	각 기능마다 대안 1개 이상과 복잡도 비교 (표 참고)

인터뷰 (25%)	5단계 답변(DS - > 이유 - > 코드위치 - > 대안 - > 복잡도) 팀원별 연습
라이브 데모 (20%)	AI+X Studio PC에서 시작 - > 플레이 - > 게임오버까지 무crash 실행

5. 데모 전 남은 작업

- 게임오버 / 클리어 화면 — player.die()가 빈 함수, 보스 처치 후 종료 처리 필요
- 리더보드 게임오버 화면 연결 — 저장/로드(leaderboard.py)는 있으나 게임오버에서 호출 경로 미연결
- 코드 정리 — enemy_boss.py(중복) 제거, [DEBUG] 키(K_b/K_t/floor=5 시작) 제거
- PPT가 실제 코드와 일치하는지 최종 점검

핵심 메시지: Run it / Defend it / Compare it — "돌아간다"가 아니라 "왜 이 자료구조를 골랐는가"를 말할 수 있어야 한다.